CS 4177 Software Vulnerabilities

Software Reverse Engineering+
Dynamic Reversing Part 1

Developed by Dr. Jaime C. Acosta

This training exercise is meant to be completed individually.

Tools: IDA Pro 7 (Free)

IDA Pro is a very widely used tool for software reverse engineering. It has static and dynamic analysis capabilities and has a great library of known code signatures to find and match, e.g., common C, C++, .NET code and then label them as such. You will learn to use IDA Pro and especially its dynamic analysis capabilities to uncover the secret plans of the evil Shinra Corporation.

Note: Questions with a * have an associated challenge (worth points) in the CIT system.

Mission Briefing – Uncovering and exposing Shinra

Barret, your boss, has just informed you that the AVALANCHE rebellion group has received a portion of old intel that you can leverage to learn about Shinra's plans to continue to build Mako reactors, which are hurting the planet. The problem is that the data is encoded and nobody else has the skillsets to uncover the hidden plans. You need analyze the old intel to learn where the updated servers are located. Afterward, you'll have to reverse engineer binary files to obtain deeper access.

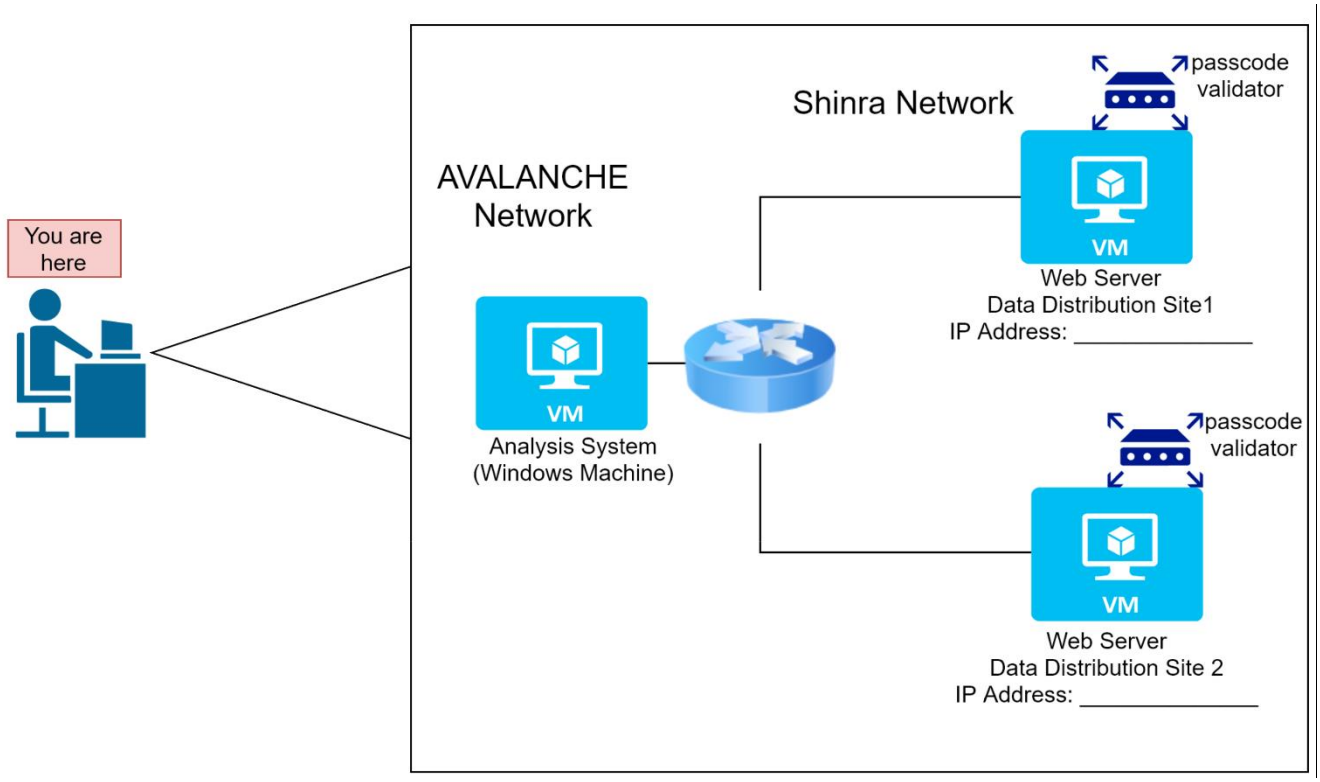


Figure 1 Network Scenario

In Summary, your job is to complete the following tasks.

1. Learn the basics of ***dynamic reverse engineering*** by creating and analyzing your own code.
2. Reverse engineer an old file that was captured by AVALANCHE and then obtain critical information from Shinra.
3. Use concolic code execution to automate the analysis and passcode retrieval of a binary.

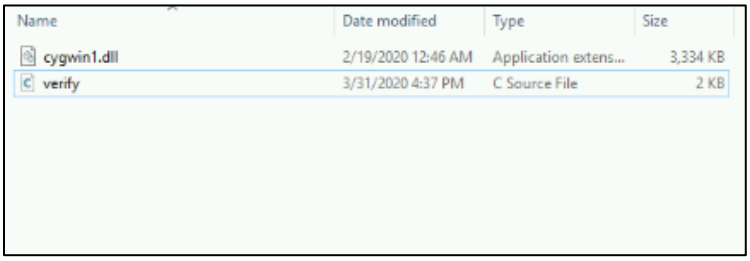
Step 1 – Dusting off your shoulders – From high- to low-level instructions

Create a high-level C program and then analyze the resulting binary file.

P1. If you haven't already, connect to the Collaborative Innovation Testbed (CIT) with the credentials provided to you by the professor. When you login, you'll notice that you'll land in a ***Windows 10*** machine. If the screen is black, press the **Enter** key.

1. If you're presented with a login screen, use the following credentials to get in:
   username: **cyber**
   password: **arlsouth**

2. We'll start by completing some high-level language template code. The code is written in the C language, but don't worry if you're not familiar with it; the following steps will walk you through the changes you need to make.
   Open the STEP 1 folder on the Desktop and double click on the C code file named **verify**



   This will start the ***Visual Studio Code*** application.

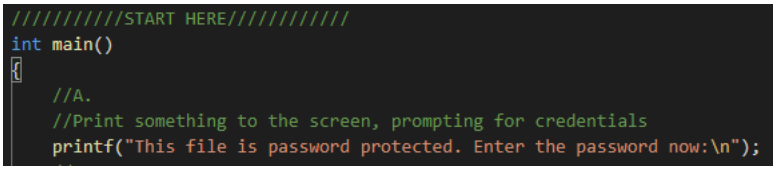
3. *Look at the code below the text `int main()` and try to understand it. Below is a list of comments in no particular order. In the code file, you'll notice the letters A., B., … G. In the table below, do your best to write the letter indicating where the comments go in the code.
   The first entry is filled in for you.

   If you get stuck, you can ask the instructor for help.

Table 1 High Level Code Comments

Location in code (letters A-G)	Comment
	Create storage location called "input" to hold the user-entered data
	Compare user input with the correct password
	Read input from the User and store it in the input storage location
	Go here if the input matches the password
A.	Print something to the screen, prompting for credentials
	Go here if the input does not match the password
	Create storage location called "result" to hold whether user-entered data matches the secret password

4. Now go back to the code file and replace the text **REPLACE ME** with the comment in the previous table (keep the //). Here's a screenshot showing an example of the first comment.



5. Spend a bit of time convincing yourself that your comments make sense (it may help to just read the comments from A-G in order).
6. Now let's ***compile*** the code, converting it into something that the computer can read: a ***raw binary*** file. On the Windows desktop, double-click on the Icon for Cygwin 

7. Execute the following command to jump into the directory where you have your code file.
   cd "STEP 1"
   Note: quotes are needed here since the directory has a space

8. Compile the program using the well-known GNU C Compiler (gcc).

gcc verify.c -o verify.exe -O0

name of code file — **output name** — **optimize zero**

9. Run the program (this may take 10-20 seconds to start).
./verify.exe

10. The text should be familiar. It's the printf output (label **A.** in the code). Recall that **label B.** in the code asks for user input; so go ahead and type something and press the Enter key.

At this point, it doesn't matter what you input as the code is still missing something to handle the match/unmatch for the ***password***. Now that we know our code is working correctly, let's add a notification to the user depending on their input.

11. Go back to ***Visual Studio Code.*** Focus on the code section labeled **E**. As you wrote in your comment, this code compares the user's input with the character string **MY_PASS\n**. Replace MY_PASS with your name instead (leave the \n); in all lowercases.

The reason for the "\n" is because this is the character that represents the "Enter" keypress (called a newline) from the terminal window.

12. Now jump down to the ***if*** statement (labeled **F.**). Let's add a printf statement that will indicate to the user that they provided the correct password. Place it below the **//1.** as shown here.

```
if(result == 0) //0 means "TRUE" anything else means "FALSE"
{
    //Fill in this section with a printf
    //1.
    printf("You are granted access!");
}
```

13. Now add a similar print statement below the ***else*** statement (labeled **G.**). Add a ***printf*** statement below the **//2.** that will indicate to the user that they did not provide the correct ***password***. Save your file by clicking on **File->Save** or by pressing **Ctrl+s**.

14. Go back to your Cygwin window and **compile** the code again (use the same command as before, in number 8).

15. Run the program (as you did in number 9) and try a valid input (your name). Afterwards, retry with an invalid input (something like "Shinra is cool").

16. If you receive the expected output, then you can go on, otherwise, review the previous steps and make sure you don't have any typos. Ask the instructor for help if you're stuck.

Step 2 – Reflection… in Reverse – IDA and assembly basics

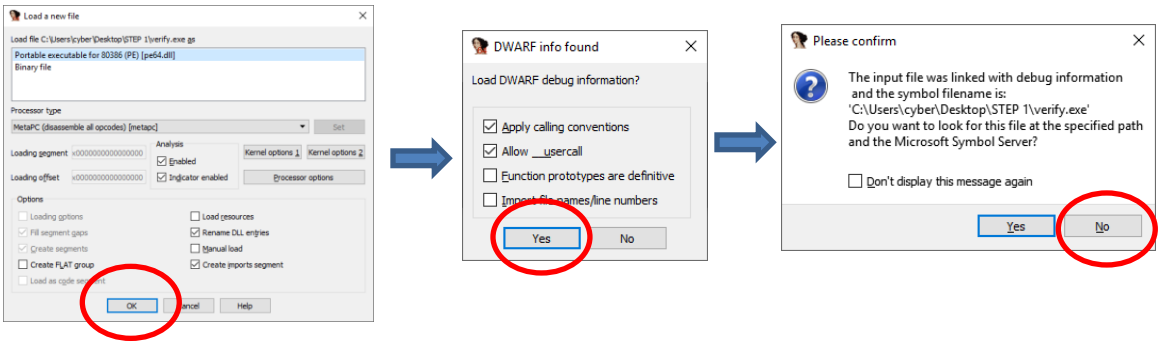
Now it's time to dive in and look at the low-level computing instructions that our C code was compiled into. You've converted the C code into something called assembly (and in particular, **x86** instructions). Now let's see what these look like with a ***disassembler*** known as IDA Pro**.**

To help with the later parts of this exercise, we'll make it our goal to **reverse engineer** the ***password*** that we set in our program so that we get the "success" message.

1. Double click on the IDA Pro icon 

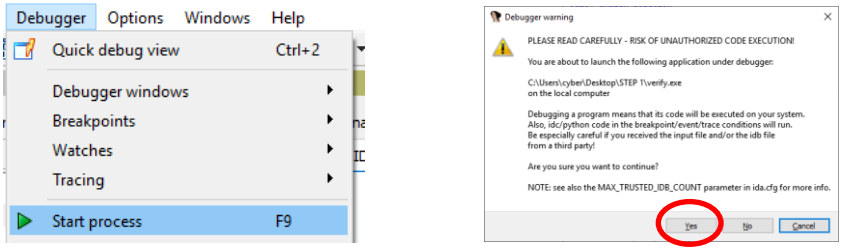
on the Desktop. Click on the **New** button and then ***Navigate to*** and ***Select*** the **verify** file in your **Desktop/STEP 1** folder.

2. In the next window, just leave as default and click **OK** and then **YES** and then **No** as shown below.

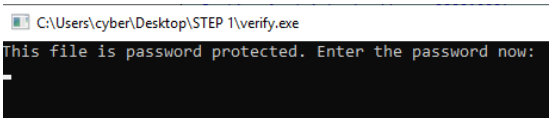


After a short while you'll see the starting point of the assembly program. It's very easy to get overwhelmed at this point because you're seeing all of the instructions, environment variables, and memory segmentation parameters, among many many others. The key is to identify ***what*** is of interest at this point.

3. Let's first run our program using the debugging capabilities of IDA Pro. Click on the **Debugger** menu item at the top of the IDA Pro window and then select Start Process (Shortcut is F9). Afterwards, please **YES** on the popup.



If done correctly, then you should see a terminal window pop up prompting you for a password.



4. Enter some input (doesn't matter what) and press **Enter**. The Window will close and you'll return back to the IDA Pro environment.

****

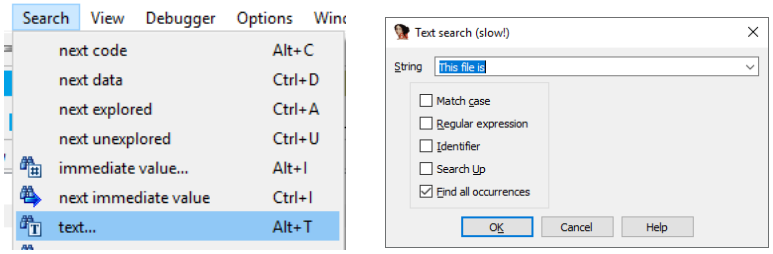
Debugging is a mechanism used by programmers to allow them to analyze state of the machine while a program runs. Some of the most commonly used features include ***variable watching*** (looking at values stored in different areas of memory and how they change), setting ***breakpoints*** (stopping execution at certain points of the program), and ***step-by-step execution*** (allowing single or batches of instructions to execute at the analyst's discretion).

You'll go through all three of these in the following steps.

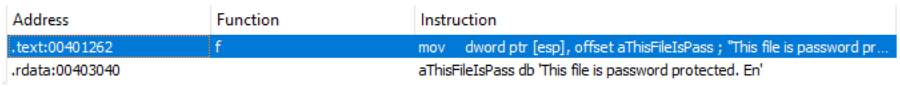
****

Let's first find a place in the binary that we can match up with the instructions that you wrote in the C program. A good place to start is to look for the text that appears in the terminal pop up window.

5. Click on the **Search** menu item and then select **Text**. Then enter a portion of the text that gets ***printed*** onto the window when verify.exe prompts for a password. Select the ***Find all occurrences*** box and then press the OK button.



6. *You'll notice that there are two ***addresses*** where this string occurs in the entire program. The first is in the ***.text*** section (where instructions go) and another in the rdata section (where read-only text, AKA text that cannot be change during the program, goes).



Double-click on the first entry and you'll be taken to the area of assembly code where the string is used.

****

The assembly language that you are viewing, called **x86**, uses one of two display styles. The one that you see in IDA Pro is called ***Intel*** style. The basic format of ***storage*** instructions is as follows:

Operation, **Destination**, **Source**

Operation – AKA opcode, this is the action
Destination –where the result of the operation is read and/or stored
Source – an input that is read for the operation

Here are some examples:

a. mov eax, 2 (**move** the value 2 **into** eax)
b. add eax, 1 (**add** the value 1 to to the value in **eax;** store result **into** eax)
c. mov [ebx+4], eax (**move** the value in eax **into** *location number*: ebx+4)

You may see other keywords within this instruction form, but for this exercise, you can glean over them. For example,

d. mov dword ptr [esp+8], 0
can be thought of as
mov [esp+8], 0 (**move** the value 0 into location number: esp+8)
e. mov dword ptr [esp], offset aThisFileIsPass ; "This file is password…"
in this case, a comment, indicated with ";" is showing the text at this location
mov storageSpaceA, "This file is password…"

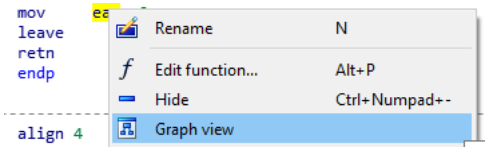
****

Right away, you should be able to find the password in your current view! You'll notice that it shows up as an auto-generated ***comment*** (the grayed-out text next to instructions). Comments are not processed by the computer and start with the ";" character. IDA Pro identified the readable text and added this comment automatically (Note: IDA doesn't know that this is a password… that's our job to find out).

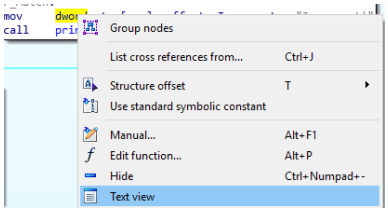
Step 3 – Slowing Down Time – IDA breakpoints and debugging

Now you'll step through the program and learn how to use IDA to add comments.

1. Right-click on any ***instruction*** and then select Text View (if you don't see **Text View**, choose **Graph View**). You can switch back and forth between these views.



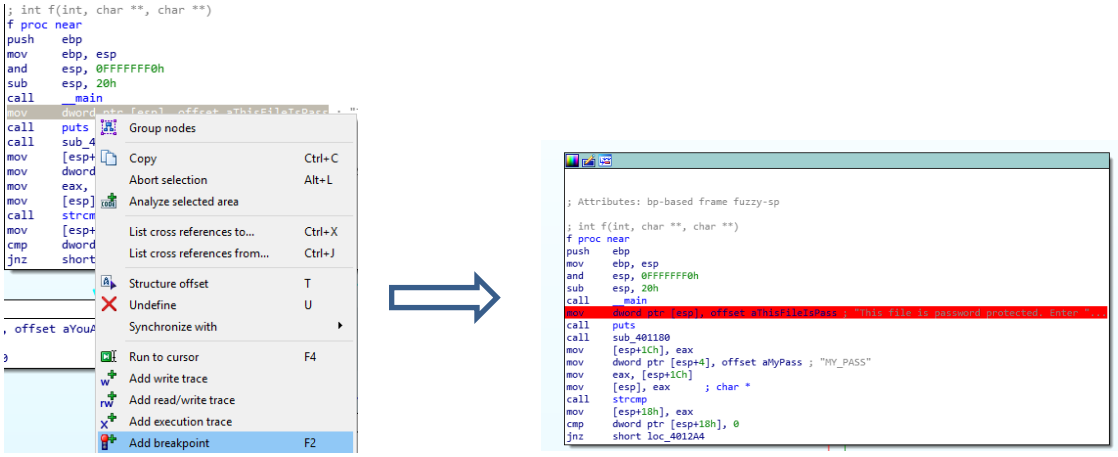
Selecting Graph View



Selecting Text View

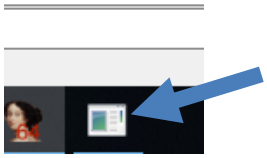
2. These are the two ways to view the assembly in IDA Pro. Switch back to the ***Graph View***. We'll use this for the rest of the exercise.

3. Let's set a ***breakpoint*** to signal to the program to stop at a particular instruction. Right-click on the line with the **dword ptr [esp], offset aThisFileIsPass…** it's immediately below the call to __main. Select ***Add breakpoint*** as shown below. The line should turn red.



4. Now ***debug*** the program as you did before (F9) and notice that the line that was previously red is now purple. The program is frozen in place here.

5. Find the terminal window on the task bar at the bottom of your screen and click on it.



The terminal window is blank; text has not yet been written to the screen.

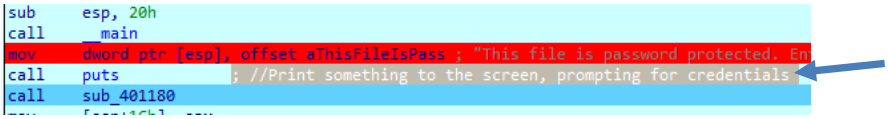
6. Now we'll step through a few instructions. In the IDA Pro Window, under the Debugger menu item, select ***step over*** (shortcut is F8). Look back at the terminal window.

7. *Keep repeating this process until you notice that the text appears in the terminal window. Write the **instruction** that is executed to make the text "This file is password protected…" appear in the terminal:

_____ _____

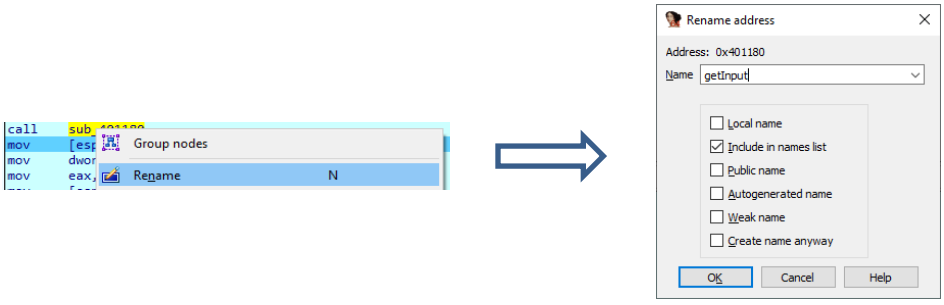
You just found a "function call" instruction. A function is a block of code that executes some behavior and then when finished, returns back, sometimes but not always, with a result. Recall the printf(...) in your C code – ***printf*** is a function to write to the terminal screen.

8. Click on the line with the instruction you found in 7 and then press the ":" key on the keyboard. Enter the same comment you used to label **//A.** in your high-level code. (Table 1 Label A) See the Figure below.

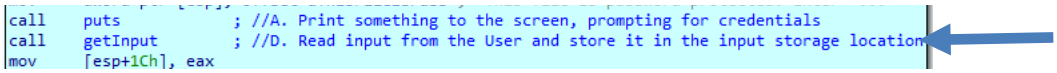


9. Now **step over** the next few instructions. Recall that F8 is the shortcut key for this. Eventually, you'll notice that the purple bar disappears, and you can't move on. This is because the terminal window is waiting for input.
   Open the terminal window and type the string: "Shrina rules" and press enter.

10. Back in IDA, you'll see that the program has continued. Right click on the **sub_401180** function label and select **rename.** Call it **getInput**.



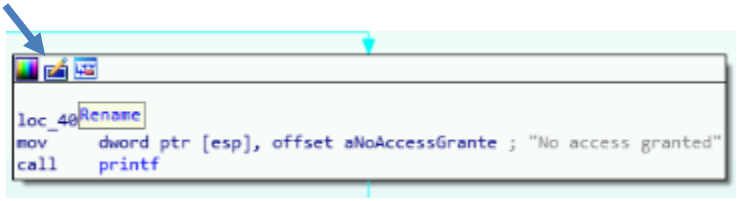
11. Add the comment labeled **D.** from Table 1 (recall that the shortcut to add is ":").



12. *Keep ***stepping over*** instructions **until** the **jnz** (or jump) instruction is highlighted. Jump instructions are like forks in the road. Depending on some conditions, the program will either go left or right. Look at the code in each of the ***forks***.
    Based on your input ("Shinra rules"), which path do you think the program will take, left or right?
    _____

13. Step over once more and confirm your answer to number 12. If you don't understand why this was the case, please ask the instructor.

14. Add a ***memory label*** for the path that ***is taken*** when the input ***does NOT match*** the ***password*** clicking on the ***rename*** icon as shown below and entering the text: **G_NoPassMatch** in the code block as shown below.



15. Let's be complete; add a ***memory label*** with the text: **F_PassMatch** for the path that ***was NOT*** taken (the case where the input would have been correct).
16. Let the program run its course by selecting the ***Debugger*** menu item and selecting ***Continue process***.

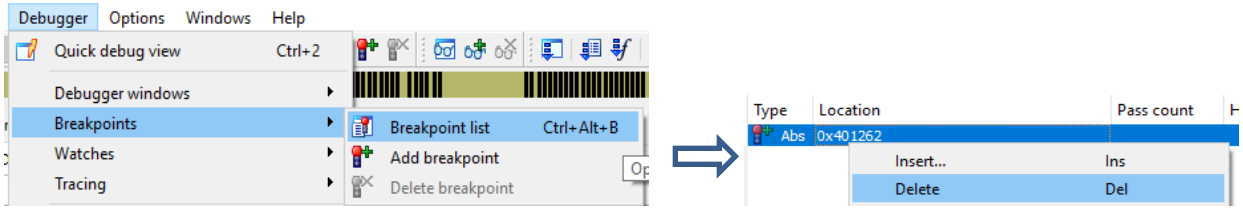
****

Functions are special in that sometimes they take as input some values (remember you passed some text to the printf function. These input values are usually ***set up*** immediately before the function call. Look back at your line where you set your break point. Notice that it is immediately before the **call puts**, which writes to the screen.

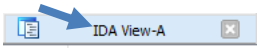
****

17. Find the **call** to the **strcmp** function (recall that you used this in your C code) and add the comment that you used in your C code (comment **E.** in Table 1). The call to the **strcmp** function is three instructions before the jnz you found earlier.

18. It turns out that the ***mov*** instructions before the **call strcmp** are setting up the **two** inputs (the two items that will be compared, just like in your C code). Remove your previous breakpoint by clicking on the **Debugger** menu item and selecting **Breakpoints->Breakpoint list**… Press delete on the entry to remove the breakpoint.



19. Click back on the IDA View tab.



20. Add a breakpoint at the call to strcmp and then debug your program once again until you reach the breakpoint (enter "Shinra rules" in the terminal as the input to reach the breakpoint.)

21. *Now that the program is again frozen, hover your mouse over eax; Does this register contain user-input or the stored/hard-coded password?

    _____

22. *By looking at the assembly instructions (focus on the auto-generated comments) and recalling the standard structure of the mov instruction **(mov destination, source)**
    where is the other text input for ***strcmp*** stored?
    Note: recall that you can ignore additional keywords like dword, ptr, for now.
    [_____+___]

23. Let the program finish by pressing F9 (Shortcut for **Debugger->Continue process**).

Great! Now you have an idea of how this program checks if the user input matches the password. We only have two labels left to finish from Table 1 (creating the storage space for input and result). In the assembly it turns out that these two are done in one swoop; using a single sub instruction.

****

x86 uses the concept of a ***stack*** (it works much like a stack of Cloud's favorite food (probably), pancakes. As they're cooked, they're placed on top of each other. To consume, one is taken off the top). The stack is used to hold local storage ***variables*** (like the two that you created in your C code). Depending on the type of data that the variables will hold, they take up different amounts of space.

The deeper workings of the pancakes …err… stack is outside the scope of this exercise, but look into future exercises, taking a reverse engineering course, or do some additional self-learning for more.

****

24. Add the final comment to the sub instruction to complete the commenting of the assembly.

```
and     esp, 0FFFFFFF0h
sub     esp, 20h        ; //B. Create storage location called "input" to hold the user-entered data
call    __main
```

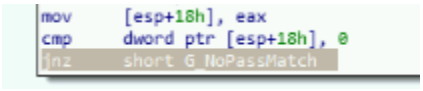
25. Save the file by clicking on **File->Save As**. In the default folder, name the file ***annotated*** and click the **Save** button.

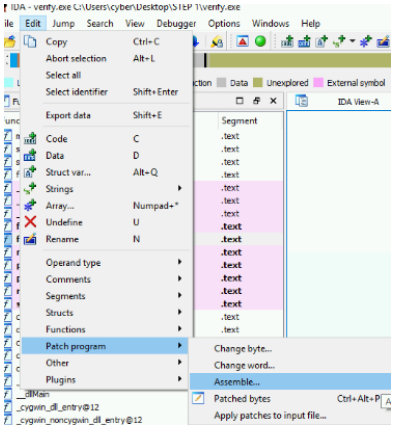
Step 4 – The Power of Control – Modifying Binaries

In this step, you'll use a feature called patching. This is when you, first, modify the IDA Pro database file, which is what you're working with in IDA at any given moment; and then write these changes to the original binary.

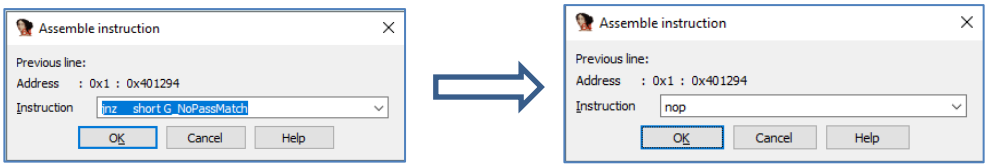
1. Again, save a copy of your file and now name it ***annotated_mod***.

2. We want to modify the program so that we always take the *validated* fork/branch/path. To do this, highlight the **jnz short G_NoPassMatch** instruction. Click on the **Edit** menu item and select **Patch program**-> **Assemble…**





3. We want to modify the **jnz** instruction (which stands for jump not zero – which can be interpreted here as ***jump if not match***) into an instruction that does nothing (called a no-operation or nop for short).
   Change the original instruction into a **nop** and press the OK button – ***Only do this once!***
   You'll get another pop-up. Press cancel.



4. You'll notice a new residual instruction appears: ***push cs***. This is due to the fact that the nop instruction is shorter than the original; so some leftover bytes are still here. Replace this instruction with another ***nop***. Notice that the graph has now changed; there no path now to the **G_NoPassMatch** code!

5. Debug the program again with IDA and convince yourself that you've made the modifications.

6. Now let's write these changes to a binary. Click on the **Edit** menu item and select

   Patch program-> **Apply patches to input file** and then press OK.

7. Go back to your Cygwin window and try running program again. If you did everything correctly, your input won't matter; you'll always be an authenticated user!

****

Uber Challenge: Use the IDA Pro patch feature to change the behavior of the verify.exe program so that the program **never asks for input**; it skips this step and always assumes the user is authenticated.

****

Great! You've reversed the program, commented your high-level and low-level code, and learned some IDA Pro basics. You've completed the exercise and ready for Part 2.

Deliverables: Email a document file with the subject **Assignment-DynReverse1** to svutep@gmail.com
The document must contain the following:

- A one-two paragraph write-up including (1) the purpose of the exercise, (2) where you had trouble (if any), (3) highlights about what you learned.